GENERALIZATION, REGULARIZATION, AND KNOWLEDGE RETENTION IN NEURAL NETWORK ARCHITECTURES

Victor Vargas Miranda
UCENFOTEC Costa Rica
vvargasm@ucenfotec.ac.cr

ABSTRACT

This investigation studies generalization, regularization and catastrophic forgetting in neural networks. It also reviews how transfer learning can reuse previous knowledge and proposes an evaluation pipeline to detect overfitting and knowledge loss.

1. INTRODUCTION

Neural networks can learn complex tasks, but they may overfit the training data or forget previous knowledge when learning a new task. Generalization and regularization help control these problems. Transfer learning reuses knowledge from a pre-trained model, but fine-tuning can modify important weights. This investigation reviews these concepts and proposes a simple pipeline to evaluate new-task learning, generalization and previous-task retention.

2. GENERALIZATION AND REGULARIZATION MECHANISMS

2.1 Theoretical Framework: Risk and Generalization

Generalization aims to minimize the expected risk (L), which is the loss over the unknown ground truth data distribution P. Because P is unknown, we minimize the empirical risk ($\hat{L}$) on a finite training set D. Regularization modifies this objective to bridge the gap between these two risks:

$$\arg\min \frac{1}{|D|} \sum_{(x_i,t_i)\in D} E(f_w(x_i), t_i) + R(\dots)$$

where E is the error function and R is the regularization term.[1]

2.2 Weight Penalties: L1 (Lasso) and L2 (Weight Decay)

Weight penalties are loss function modifications that add a regularization term $R(w)$ to the empirical risk objective. This term is independent of the targets t and is used to provide inductive bias by penalizing complex models.

L1 Regularization (Lasso)

L1 regularization, often referred to as Lasso (Least Absolute Shrinkage and Selection Operator), adds the L1 norm of the weights as a penalty.

- **Mathematical Objective:** $R(w) = \lambda\|w\|_1 = \lambda \sum_i |w_i|$
- **Mechanism:** Because the absolute value function has a constant gradient (except at zero), the penalty exerts a constant pressure on weights toward zero regardless of their magnitude.[1]
- **Effect on Topology:** Lasso is known for promoting sparsity. It often drives the weights of less important features to exactly zero, effectively performing automatic feature selection. This produces a more interpretable and compact model.

L2 Regularization (Weight Decay)

L2 regularization, or Weight Decay, adds the squared L2 norm of the weights to the loss function.

- **Mathematical Objective:** $R(w) = \frac{\lambda}{2}\|w\|_2^2 = \frac{\lambda}{2}\sum_i w_i^2$
- **Mechanism:** The penalty is proportional to the square of the weight values. This means larger weights are penalized much more heavily than smaller ones, favoring a "simpler" model where the mapping is smooth.[1]
- **Bayesian Interpretation:** L2 regularization corresponds to assuming a Gaussian prior on the weights: $p(w) = \mathcal{N}(w \mid 0, \lambda^{-1}I)$.
- **Effect on Optimization:** It prevents any single weight from becoming too "extreme" or dominant. In the context of deep learning, weight decay is widely used but is sometimes found to be neither necessary nor sufficient for generalization, as large networks can still perfectly fit (memorize) random data even when L2 is active.

2.3 Dropout

Dropout is primarily a network topology modification that acts stochastically during training.

- **Mechanism:** It randomly "drops out" units (and their connections) with probability $1 - p$, creating a "thinned" network for each training case. This prevents units from co-adapting—where a neuron only performs well in the presence of specific other neurons.[3]
- **Mathematical Analysis:** The feed-forward operation for a layer l becomes:

$$r^{(l)} \sim \text{Bernoulli}(p)$$

$$\tilde{y}^{(l)} = r^{(l)} * y^{(l)} \quad \text{(element-wise product)}$$

$$z_i^{(l+1)} = w_i^{(l+1)}\tilde{y}^{(l)} + b_i^{(l+1)}$$

- **Topology Modification:** Training with dropout is equivalent to training an ensemble of 2^n thinned networks with extensive weight sharing. At test time, the weights are scaled ($W_{\text{test}} = pW$) to approximate the geometric mean of the predictions of all these thinned models.
- **Loss Interpretation:** For linear regression, marginalizing dropout noise is mathematically equivalent to a modified form of Ridge Regression (L2), where the penalty scales the weight cost by the standard deviation of each data dimension.

2.4 Data Augmentation

Data augmentation is a modification via the training data that transforms the original dataset D into an augmented set D_R.

- **Mechanism:** It uses stochastic transformations (τ_θ) to generate new samples, effectively expanding the training set size.[1]
- **Mathematical Analysis:** It replaces the empirical risk with a data-augmented loss ($\hat{L}_A$) that integrates over the distribution of transformation parameters $p(\theta)$:

$$\hat{L}_A = \frac{1}{|D|} \sum_{(x_i,t_i)\in D} \mathbb{E}_\theta[\ell(\tau_\theta(x_i), t_i)]$$

- **Effect:** This process creates an augmented distribution Q. The closer Q is to the ground truth distribution P, the more effectively the augmented loss simulates the true expected risk. Data augmentation is often more powerful than loss-based regularizers because it can enforce complex symmetries (like rotation or translation invariance) directly through the data.

2.5 Early Stopping

Early stopping is a modification of the optimization procedure, specifically a termination method.

- **Mechanism:** It halts training when performance on a separate validation set begins to decline, even if the training error is still decreasing.
- **Theory:** It exploits the fact that networks typically learn general, useful concepts before they begin to "memorize" the specific noise in the training set.[1]
- **Implicit Regularization:** It is often viewed as a form of implicit regularization. In linear models, for example, shorter training times via SGD act as a regularizer by converging to the solution with the smallest L2 norm.[2]

2.6 Modern Synthesis: Sharpness and Geometry

Recent research (Source 3) suggests that generalization is tied to the geometry of the loss landscape.

- **Sharpness-Aware Minimization (SAM):** This technique modifies the optimization objective to seek flatter minima. Instead of just minimizing the loss $L_S(w)$, it solves a min-max problem to find a neighborhood with uniformly low loss[4]:

$$\min_w \max_{\|\epsilon\|_p \le \rho} L_S(w+\epsilon)$$

  This approach directly addresses the problem where standard networks have the capacity to fit even random labels perfectly (zero training error) but fail to generalize. By explicitly penalizing "sharp" minima—where small parameter changes lead to large loss increases—SAM improves generalization across diverse tasks.

3. THE PROBLEM OF CATASTROPHIC FORGETTING

Catastrophic forgetting (or catastrophic interference) is the phenomenon where a neural network abruptly loses the knowledge of previously learned tasks as it incorporates information relevant to a new task. This occurs because the network weights that were optimized and vital for a previous task (Task A) are drastically overwritten to meet the objectives of a current task (Task B).[5]

3.1. The Mechanics of Weight Overwriting

The mathematical and physiological roots of this problem lie in how artificial systems manage plasticity compared to biological ones.

Probabilistic Framework: From a Bayesian perspective, learning involves finding the most probable parameter values (θ) given training data (D). For two sequential tasks, D_A and D_B, the posterior probability is:

$$\log p(\theta \mid D) = \log p(D_B \mid \theta) + \log p(\theta \mid D_A) - \log p(D_B)$$

Standard training (SGD) ignores the term representing previous knowledge ($\log p(\theta \mid D_A)$), moving the parameters toward a new local minimum that minimizes Task B's loss alone. Because networks are over-parameterized, there are many configurations that result in high performance; however, without constraints, the parameters drift out of the "low error" region for Task A.[5]

Physiological Contrast: The mammalian brain avoids this through synaptic consolidation. When a new skill is learned, a proportion of excitatory synapses are strengthened—manifesting as enlarged dendritic spines—and rendered less plastic. These stable synapses persist even as the brain learns new tasks, providing a mechanism for long-term retention that standard neural networks lack.[5]

3.2. Regularization-based Mitigation: Elastic Weight Consolidation (EWC)

EWC mimics synaptic consolidation by slowing down learning on weights identified as important for previous tasks.

Mechanism: It implements a quadratic penalty (acting like a spring) that anchors parameters to their old values. The importance of each weight is determined by the diagonal of the Fisher information matrix (F), which is equivalent to the second derivative of the loss near a minimum.[5]

The EWC Loss Function:

$$L(\theta) = L_B(\theta) + \sum_i \frac{\lambda}{2} F_i \big(\theta_i - \theta^*_{A,i}\big)^2$$

Here, λ sets the importance of the old task relative to the new one, and F_i provides the "stiffness" of the spring for parameter i.

3.3. Replay Buffers and Rehearsal Strategies

Replay involves complementing new task data with representations of old tasks to "remind" the network of what it has already learned.[6]

- **Exact Replay (Exemplars):** Strategies like iCaRL store a small subset of actual images (exemplars) from old classes. It uses a herding algorithm to select exemplars whose average feature vector best approximates the true class mean. Classification is then performed using a nearest-mean-of-exemplars rule in the feature space rather than raw network outputs.[7]
- **Generative Replay:** Instead of storing raw data (which may be limited by memory or privacy), Deep Generative Replay (DGR) trains a separate generative model—such as a Variational Autoencoder (VAE)—to produce "pseudo-data" representing the distribution of previous tasks.[6]
- **Response Preservation (Distillation):** Learning without Forgetting (LwF) uses the new task images to record "responses" from the original network for old tasks. It employs a Knowledge Distillation loss to ensure the new network's outputs for old tasks remain similar to the original ones, effectively using the new data as a proxy for the old.[8]

3.4. Task-Specific and Architectural Strategies

These approaches prevent forgetting by isolating task-specific information or growing the network's capacity.

Context-dependent Gating (XdG): For each task, a random subset of units in the hidden layers is gated (set to zero). This forces the network to use different "sub-networks" for different tasks, significantly reducing the overlap and subsequent overwriting of critical weights.[6]

Architectural Modification:

- **Feature Extraction:** The shared parameters (θ_s) are frozen, and only new task-specific layers (θ_n) are trained. While this prevents forgetting, the shared features cannot adapt to the new task.[8]
- **Progressive Neural Networks:** This strategy grows the network by adding new "columns" for every task, keeping the old weights entirely untouched.
- **Network Expansion:** Adding nodes to existing layers (similar to Net2Net) allows for new-task-specific information in early layers while preserving the original structure.[8]

3.5. Evaluation Context: The Three Scenarios

The effectiveness of these strategies depends heavily on the continual learning scenario:[6]

- **Task-IL:** Task identity is provided at test time; models can use task-specific components.
- **Domain-IL:** Task identity is not provided, but the model only needs to solve the task at hand.
- **Class-IL:** The model must solve the task and infer the task identity (e.g., which class among all seen so far). Regularization-based methods like EWC often fail in Class-IL, where replay-based approaches are currently considered necessary for acceptable performance.

4. TRANSFER LEARNING AS AN ENGINEERING STRATEGY

The use of pre-trained models in transfer learning is driven by the hierarchical nature of deep neural networks, where knowledge gained from a data-rich source domain is repurposed for a target domain to overcome data scarcity and reduce training time.[9]

4.1. Feature Extraction and Fine-Tuning

Transfer learning typically involves two main strategies: feature extraction and fine-tuning.

- **Feature Extraction:** This approach treats the front layers of a pre-trained network as a versatile feature extractor. The early layers identify high-level, general features of the data, while subsequent layers interpret this information to make specific decisions. In practice, the first n layers are copied from a base network and "frozen," meaning their connection parameters are not updated during training on the target task.[10]
- **Fine-Tuning:** In this strategy, the copied layers are not frozen but are allowed to be updated through backpropagation on the target dataset. Fine-tuning can significantly improve performance, especially if the target dataset is large enough to avoid overfitting. Interestingly, initializing a network with transferred features can boost generalization performance even after extensive fine-tuning, an effect that lingers far longer than expected.[10]

4.2. Layer Transferability: General vs. Specific

A critical discussion in transfer learning is determining which layers are transferable and where the transition from general to specific knowledge occurs.

- **Generality of Early Layers:** Early layers (such as the first and second layers in convolutional neural networks) tend to learn general features—like Gabor filters or color blobs in image tasks—that are applicable to many datasets and tasks regardless of the specific cost function used. These layers transfer almost perfectly between even dissimilar tasks.
- **Specificity of Deeper Layers:** As one moves toward the output layer, features become increasingly specific to the original task. For example, in classification, the last-layer units are specialized to the particular classes of the source dataset.
- **Barriers to Transferability:** Beyond representation specificity, transferability is hindered by fragile co-adaptation. This refers to the complex ways neurons on neighboring layers interact; when a network is "split" by freezing some layers and retraining others, the upper layers may fail to rediscover the necessary interactions with the frozen lower layers. This optimization difficulty is often most pronounced in the middle layers of a network.[10]

4.3. Transfer Learning Architectures

Deep transfer learning can be categorized into four primary architectural techniques:

1. **Network-based:** This is the most common form, involving the reuse of partial network structures and parameters pre-trained in a source domain. It assumes that the front-end features are versatile across domains.
2. **Mapping-based:** This architecture maps instances from both the source and target domains into a new data space. The goal is to make the distributions of the two domains more similar, allowing a single deep network to process both effectively.
3. **Adversarial-based:** Inspired by GANs, this approach uses an adversarial layer to find representations that are discriminative for the main task but indiscriminate between the source and target domains.
4. **Instances-based:** This technique uses specific strategies to re-weight instances in the source domain, selecting those most similar to the target domain to supplement the training set.[9]

4.4. Modern Efficient Architectures: LoRA

In the context of Large Language Models (LLMs), full fine-tuning is often prohibitively expensive. Low-Rank Adaptation (LoRA) is a modern architecture that freezes the pre-trained weights and injects trainable rank decomposition matrices into the Transformer layers. This reduces the number of trainable parameters by up to 10,000 times. Unlike other adapter methods, LoRA introduces no additional inference latency because the low-rank matrices can be merged with the original weights during deployment. Research suggests that LoRA works by amplifying specific feature directions that were learned during pre-training but not emphasized in the original general model.[11]

5. PIPELINE PROPOSAL AND PREDICTIVE EVALUATION

The purpose of this evaluation pipeline is to check three basic results when transfer learning is applied:

- The model learns the new task.
- The model does not overfit the new training data.
- The model does not lose too much knowledge from the previous task.

The pipeline should first compare two configurations already described in the investigation:

1. **Feature extraction:** the transferred layers remain frozen.
2. **Fine-tuning:** the transferred layers are allowed to change during training.

Feature extraction is used as the basic comparison because the original transferred layers are not modified. Fine-tuning can adapt better to the new task, but it can also produce overfitting or catastrophic forgetting.

5.1 Evaluation Metrics

The main measurements should be recorded after every training epoch:

- Training loss of the new task.
- Validation loss of the new task.
- Validation loss of the previous task.
- Difference between training and validation loss.
- Change in the previous-task validation loss.

Loss is selected as the main metric because the investigation explains learning, generalization, early stopping and catastrophic forgetting using training, validation and task loss. It can also be used for both feature extraction and fine-tuning.

5.2 Training and Validation Curves

For the new task B, the pipeline should calculate the generalization gap:

$$G_B^{(e)} = L_{\text{val},B}^{(e)} - L_{\text{train},B}^{(e)}$$

In this formula, $L_{\text{train},B}^{(e)}$ is the training loss and $L_{\text{val},B}^{(e)}$ is the validation loss at epoch e.

The pipeline should create a table like the following:

Epoch	Training Loss	Validation Loss	Generalization Gap
1	$L^{(1)}_{\text{train},B}$	$L^{(1)}_{\text{val},B}$	$G^{(1)}_{B}$
2	$L^{(2)}_{\text{train},B}$	$L^{(2)}_{\text{val},B}$	$G^{(2)}_{B}$
...	...	...	...

The training and validation curves should be interpreted in a simple way:

Training Loss	Validation Loss	Interpretation
Decreases	Decreases	The model is learning and generalizing.
Decreases	Stabilizes	Overfitting may be starting.
Decreases	Increases	The model is overfitting.
Remains high	Remains high	The model is not adapting enough.

These curves are preferred over training loss alone because a low training loss does not prove that the model performs well with unseen validation data.

5.3 Previous-Task Retention

Before training on the new task, the model should be evaluated using the previous-task validation set. This result becomes the retention baseline.

For a previous task A, the change in validation loss can be calculated as:

$$F_A^{(e)} = L^{(e)}_{\text{val},A} - L^{(0)}_{\text{val},A}$$

Here, $L^{(0)}_{\text{val},A}$ is the validation loss before transfer learning and $L^{(e)}_{\text{val},A}$ is the validation loss after epoch e of the new task.

Result	Interpretation
$F_A^{(e)} \approx 0$	Previous knowledge remains stable.
$F_A^{(e)} > 0$	Previous-task performance has become worse.
$F_A^{(e)}$ increases repeatedly	Forgetting is increasing during training.
$F_A^{(e)} < 0$	Previous-task validation performance improved.

This measurement is useful because catastrophic forgetting is detected by checking whether the model performs worse on the previous task, not only by checking whether its weights changed.

5.4 Combined Evaluation Matrix

The pipeline should combine the new-task curves with the previous-task validation loss.

New-Task Training Loss	New-Task Validation Loss	Previous-Task Validation Loss	Diagnosis
Decreases	Decreases	Stable	Successful transfer
Decreases	Increases	Stable	Overfitting
Decreases	Decreases	Increases	Catastrophic forgetting
Decreases	Increases	Increases	Overfitting and forgetting

This matrix separates the two problems. Overfitting is detected using the difference between training and validation behavior on the new task. Catastrophic forgetting is detected using the validation loss of the previous task.

5.5 Task-by-Checkpoint Matrix

When the model learns several tasks, the pipeline should save the validation loss of each task after every training stage.

Evaluated Task	After Task A	After Task B	After Task C
Task A	$L_{A,A}$	$L_{A,B}$	$L_{A,C}$
Task B	—	$L_{B,B}$	$L_{B,C}$
Task C	—	—	$L_{C,C}$

The diagonal shows the validation loss when each task was learned. The values to the right show how the performance changed after later tasks were learned. This matrix helps identify which task was forgotten and when the deterioration started.

5.6 Automated Validation Pipeline

The automated pipeline should follow these steps:

1. **Separate the data:** Keep a training and validation set for every task.
2. **Create a baseline:** Before transfer learning, evaluate the model on the new task and on all previous-task validation sets.
3. **Train the new task:** Apply feature extraction or fine-tuning.
4. **Evaluate every epoch:** Record the new-task training loss, new-task validation loss and previous-task validation loss.
5. **Detect overfitting:** Create a warning when training loss decreases but validation loss increases.
6. **Detect forgetting:** Create a warning when the previous-task validation loss increases compared with its baseline.
7. **Apply early stopping:** Stop training when the new-task validation loss begins to become worse while training loss continues decreasing.

8. **Save checkpoints:** Keep the model checkpoints that provide good validation performance without producing unacceptable forgetting.
9. **Compare the strategies:** Compare feature extraction and fine-tuning using the same validation sets and evaluation measurements.

5.7 Final Model Selection

The final model should not be selected only because it has the lowest training loss. The selected checkpoint should have:

- A low validation loss on the new task.
- A controlled difference between training and validation loss.
- A previous-task validation loss close to its original baseline.

Feature extraction should be preferred when fine-tuning produces only a small improvement on the new task but causes a large loss of previous knowledge.

Fine-tuning should be preferred when it improves the new-task validation loss and the previous-task validation loss remains stable.

Therefore, the proposed pipeline selects the transfer-learning configuration that gives a reasonable balance between learning the new task, generalizing to validation data and retaining the previously learned task.

6. CONCLUSIONS

1. I learned that generalization and regularization are related to catastrophic forgetting because both depend on controlling how the model changes its weights. Regularization mechanisms help the model avoid memorizing the training data, while methods such as EWC protect important weights from being changed too much when a new task is learned.

2. I learned that transfer learning does not automatically prevent catastrophic forgetting. Feature extraction reduces this problem by freezing the transferred layers, while fine-tuning allows better adaptation but can change previous knowledge. Therefore, the correct strategy depends on balancing the learning of the new task with the retention of the previous task.

3. The main challenge when designing the pipeline was evaluating overfitting and catastrophic forgetting separately. The pipeline must compare training and validation loss for the new task, while also checking the validation loss of previous tasks. The final model should not only learn the new task, but also generalize well and preserve previous knowledge.

REFERENCES

[1] Kukačka, J. Golkov, V. and Cremers, D. 2017. Regularization for Deep Learning: A Taxonomy. Technical University of Munich. DOI= https://doi.org/10.48550/arXiv.1710.10686.

[2] Zhang, C. Bengio, S. Hardt, M. Benjamin, R. Oriol, V. 2017. Understanding deep learning requires rethinking generalization. DOI= https://doi.org/10.48550/arXiv.1611.03530.

[3] Srivastava, N. Hinton, G. Krizhevsky, A. Sutskever, I. Salakhutdinov, R. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. University of Toronto. Journal of Machine Learning Research 15 (2014) 1929-1958. DOI= https://dl.acm.org/doi/10.5555/2627435.2670313.

[4] Foret, P. Kleiner, A. Mobahi, H. Behman, N. 2020. Sharpness-Aware Minimization for Efficiently Improving Generalization. Published as a conference paper at ICLR 2021. DOI= https://doi.org/10.48550/arXiv.2010.01412

[5] Kirkpatrick, J. Pascanu, R. Rabinowitz, N. Veness, J. Desjardins, G. Rusu, A. A. Milan, K. Quan, J. Ramalho, T. Grabska-Barwinska, A. Hassabis, D. 2017. Overcoming catastrophic forgetting in neural networks. DOI= https://doi.org/10.48550/arXiv.1612.00796.

[6] van de Ven, G. M. Tolias, A. S. 2019. Three scenarios for continual learning. DOI= https://doi.org/10.48550/arXiv.1904.07734.

[7] Rebuffi, S. A. Kolesnikov, A. Sperl, G. Lampert, C. H. 2017. iCaRL: Incremental Classifier and Representation Learning. DOI= https://doi.org/10.48550/arXiv.1611.07725.

[8] Li, Z. Hoiem, D. 2016. Learning without Forgetting. DOI= https://doi.org/10.48550/arXiv.1606.09282.

[9] Tan, C. Sun, F. Kong, T. Zhang, W. Yang, C. Liu, C. 2018. A Survey on Deep Transfer Learning. DOI= https://doi.org/10.48550/arXiv.1808.01974.

[10] Yosinski, J. Clune, J. Bengio, Y. Lipson, H. 2014. How transferable are features in deep neural networks? DOI= https://doi.org/10.48550/arXiv.1411.1792.

[11] Hu, E. J. Shen, Y. Wallis, P. Allen-Zhu, Z. Li, Y. Wang, S. Wang, L. Chen, W. 2021. LoRA: Low-Rank Adaptation of Large Language Models. DOI= https://doi.org/10.48550/arXiv.2106.09685.